JavaScript Multiple Choice Questions - Solutions Guide

Questions 51-100

This document contains comprehensive explanations for 50 JavaScript multiple choice questions covering fundamental concepts including data types, operators, control flow, and type coercion.

Question 51

console.log(typeof NaN);

- NaN
- **number** ✓
- undefined
- object

Explanation: NaN (Not-a-Number) is technically a numeric type in JavaScript. Despite its name, `typeof NaN` returns `'number'`. NaN is a special value representing an invalid number.

Question 52

Which is block scoped?

- var
- let
- const
- **let & const** ✓

Explanation: Both `let` and `const` are block-scoped, meaning they are only accessible within the block `{}` where they are defined. `var` is function-scoped.

Question 53

console.log(5 + "5");

- 10
- **55** ✓
- "10"
- NaN

Explanation: When using the `+` operator with a number and string, JavaScript coerces the number to a string and performs concatenation. `5` becomes `"5"` and `"5" + "5" = "55"`.

Question 54

Strict equality operator:

- ==
- === ✓
- !=
- =

Explanation: `===` is the strict equality operator that checks both value and type without type coercion. `==` is loose equality that performs type coercion.

Question 55

Falsy value:

- "0"
- **0** ✓
- "false"

Explanation: In JavaScript, falsy values are: `false`, `0`, `""` (empty string), `null`, `undefined`, and `NaN`. The strings `"0"` and `"false"`, and empty array `[]` are truthy values.

Question 56

Loop runs at least once:

- while
- **do-while** ✓
- for
- for-in

Explanation: `do-while` loop executes the code block first, then checks the condition. This guarantees at least one execution. Other loops check the condition before executing.

Question 57

console.log(Boolean(""));

- true
- **false** ✓
- ""
- undefined

Explanation: Empty string `""` is a falsy value in JavaScript. `Boolean("")` converts it to `false`.

Question 58

console.log(2 ** 3);

- 6
- **8** ✓
- 9
- 5

Explanation: `**` is the exponentiation operator. `2 ** 3` means $2^3 = 2 \times 2 \times 2 = 8$.

Question 59

typeof null →

- null
- **object** ✓
- undefined
- number

Explanation: This is a famous JavaScript bug from the early days. `typeof null` returns `'object'` even though `null` is a primitive value, not an object.

Question 60

Which is NOT primitive?

- string
- number
- **object** ✓
- boolean

Explanation: Primitive types in JavaScript are: `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, and `bigint`. Objects are complex data types, not primitives.

Question 61

var scope:

- block
- **function** ✓
- global only
- module

Explanation: Variables declared with `var` are function-scoped. They are accessible throughout the entire function, but not limited to blocks like `if` or `for` statements.

Question 62

console.log("5" - 2);

- **3** ✓
- "3"
- NaN
- 52

Explanation: Unlike the `+` operator, the `-` operator triggers numeric conversion. `"5"` is converted to `5`, then `5 - 2 = 3`.

Question 63

Switch uses comparison:

- ==
- === ✓
- !=
- =

Explanation: `switch` statement uses strict equality (`===`) for comparison, checking both value and type without coercion.

Question 64

Which stops loop:

- **break** ✓
- continue
- return
- exit

Explanation: `break` terminates the loop immediately. `continue` skips the current iteration and moves to the next. `return` exits the function. `exit` is not a JavaScript keyword.

Question 65

console.log(0 == false);

- **true** ✓
- false
- NaN
- error

Explanation: With loose equality (`==`), JavaScript performs type coercion. `0` and `false` are both falsy values, so `0 == false` is `true`.

Question 66

console.log(0 === false);

- true
- **false** ✓
- NaN
- error

Explanation: Strict equality (`===`) checks type and value without coercion. `0` is a number and `false` is a boolean, so they are not strictly equal.

Question 67

console.log(typeof []);

- array
- **object** ✓
- list
- undefined

Explanation: Arrays are objects in JavaScript. `typeof []` returns `'object'`. To check for arrays specifically, use `Array.isArray([])`.

Question 68

Infinite loop:

- **for(;;)** ✓
- while(false)
- do{}while(false)
- for(let i=0;i<0;i++)

Explanation: `for(;;)` creates an infinite loop because all three parts (initialization, condition, increment) are omitted, making the condition always `true`.

Question 69

console.log(+"5");

- "5"
- **5** ✓
- NaN
- undefined

Explanation: The unary `+` operator converts its operand to a number. `+"5"` converts the string `"5"` to the number `5`.

Question 70

Nullish operator:

- ||
- **??** ✓
- &&
- ?:

Explanation: `??` is the nullish coalescing operator. It returns the right operand only when the left is `null` or `undefined` (not other falsy values like `0` or `''`).

Question 71

console.log(1 && 0);

- 1
- **0** ✓
- true
- false

Explanation: AND operator (`&&`) returns the first falsy value or the last value if all are truthy. `1` is truthy, so it evaluates `0`, which is falsy and returned.

Question 72

console.log(1 || 0);

- **1** ✓
- 0
- true
- false

Explanation: OR operator (`||`) returns the first truthy value or the last value if all are falsy. `1` is truthy, so it's returned immediately.

Question 73

console.log(!1);

- true
- **false** ✓
- 0
- NaN

Explanation: NOT operator (`!`) converts the value to boolean and negates it. `1` is truthy, so `!1` converts to boolean `true`, then negates to `false`.

Question 74

Which iterates values:

- for...in
- **for…of** ✓
- forEach
- map

Explanation: `for...of` iterates over values of iterable objects (arrays, strings, etc.). `for...in` iterates over keys/indices.

Question 75

Which iterates keys:

- **for…in** ✓
- for...of
- map
- filter

Explanation: `for...in` iterates over enumerable property names (keys) of an object, including inherited properties.

Question 76

`console.log(parseInt("10px"));`

- **10** ✓
- NaN
- 0
- undefined

Explanation: `parseInt()` parses a string and returns an integer. It stops parsing when it encounters non-numeric characters. `"10px"` returns `10`.

Question 77

console.log(Math.floor(4.9));

- **4** ✓
- 5
- 4.9
- 5.1

Explanation: `Math.floor()` rounds down to the nearest integer. `4.9` rounds down to `4`.

Question 78

console.log(Math.ceil(4.1));

- 4
- **5** ✓
- 4.1
- 5.9

Explanation: `Math.ceil()` rounds up to the nearest integer. `4.1` rounds up to `5`.

Question 79

console.log(isNaN("abc"));

- **true** ✓
- false
- NaN
- error

Explanation: `isNaN()` attempts to convert the value to a number. `"abc"` cannot be converted to a valid number, so it returns `true`.

Question 80

console.log(typeof undefined);

- **undefined** ✓
- object
- null
- number

Explanation: `typeof undefined` returns `'undefined'`. This is one of the primitive types in JavaScript.

Question 81

console.log(5 % 2);

- **1** ✓
- 2
- 0
- 5

Explanation: `%` is the modulo operator that returns the remainder of division. `5` divided by `2` is `2` with remainder `1`.

Question 82

console.log(!!"hello");

- **true** ✓
- false
- "hello"
- NaN

Explanation: Double NOT (`!!`) converts a value to boolean. `"hello"` is truthy, so `!"hello"` is `false`, and `!!"hello"` is `true`.

Question 83

console.log(1 < 2 < 3);

- **true** ✓
- false
- NaN
- error

Explanation: Evaluated left to right: `(1 < 2)` returns `true`, then `true < 3`. `true` converts to `1`, and `1 < 3` is `true`.

Question 84

console.log(3 > 2 > 1);

- true
- **false** ✓
- NaN
- error

Explanation: Evaluated left to right: `(3 > 2)` returns `true`, then `true > 1`. `true` converts to `1`, and `1 > 1` is false.

Question 85

console.log(true + true);

- **2** ✓
- 1
- true
- NaN

Explanation: In arithmetic operations, `true` converts to `1`. `true + true` becomes `1 + 1 = 2`.

Question 86

console.log("5" * 2);

- **10** ✓
- NaN
- 52
- 7

Explanation: Multiplication operator (*) triggers numeric conversion. `"5"` converts to `5`, then `5 * 2 = 10`.

Question 87

console.log(5 + null);

- **5** ✓
- null
- NaN
- undefined

Explanation: In arithmetic operations, `null` converts to `0`. `5 + null` becomes `5 + 0 = 5`.

Question 88

console.log(5 + undefined);

- **NaN** ✓
- 5
- undefined
- error

Explanation: `undefined` converts to `NaN` in arithmetic operations. `5 + NaN = NaN`.

Question 89

console.log(null == undefined);

- **true** ✓
- false
- NaN
- error

Explanation: With loose equality (`==`), `null` and `undefined` are considered equal to each other (special case in JavaScript).

Question 90

console.log(null === undefined);

- true
- **false** ✓
- NaN
- error

Explanation: With strict equality (`===`), `null` and `undefined` are different types, so they are not strictly equal.

Question 91

console.log(typeof function(){});

- **function** ✓
- object
- callable
- undefined

Explanation: Functions have their own type in JavaScript. `typeof function(){}` returns `'function'`, even though functions are technically objects.

Question 92

console.log(1 ? "yes" : "no");

- **yes** ✓
- no
- 1
- true

Explanation: Ternary operator: `condition ? trueValue : falseValue`. `1` is truthy, so `"yes"` is returned.

Question 93

console.log("" ? "yes" : "no");

- yes
- **no** ✓
- ""
- false

Explanation: Empty string `""` is falsy, so the ternary operator returns the false branch `"no"`.

Question 94

console.log(0 || "hello");

- 0
- **hello** ✓
- false
- undefined

Explanation: OR operator (`||`) returns first truthy value. `0` is falsy, so it evaluates and returns `"hello"`.

Question 95

console.log(0 && "hello");

- **0** ✓
- hello
- false
- undefined

Explanation: AND operator (`&&`) returns first falsy value or last value. `0` is falsy, so it's returned immediately.

Question 96

console.log("5" == 5);

- **true** ✓
- false
- NaN
- error

Explanation: Loose equality (`==`) performs type coercion. `"5"` is converted to `5`, and `5 == 5` is `true`.

Question 97

console.log("5" === 5);

- true
- **false** ✓
- NaN
- error

Explanation: Strict equality (`===`) checks type and value. `"5"` is a string and `5` is a number, so they are not strictly equal.

Question 98

console.log([1,2] + [3,4]);

- **"1,23,4"** ✓

1,2,3,4

- 10
- NaN

Explanation: Arrays are converted to strings when using `+`. `[1,2]` becomes `"1,2"` and `[3,4]` becomes `"3,4"`. Concatenation gives `"1,2" + "3,4" = "1,23,4"`.

Question 99

console.log(typeof Infinity);

- **number** ✓
- infinity
- object
- undefined

Explanation: `Infinity` is a special numeric value in JavaScript. `typeof Infinity` returns `'number'`.

Question 100

console.log(0.1 + 0.2 == 0.3);

- true
- **false** ✓
- NaN
- error

Explanation: Floating-point precision issue: `0.1 + 0.2 = 0.30000000000000004` in JavaScript, not exactly `0.3`. This is due to binary representation limitations.

Summary of Key Concepts

Type Coercion

JavaScript performs implicit type conversion in many operations. The `+` operator with strings performs concatenation, while `-`, `*`, `/`, `%` perform numeric conversion.

Equality Operators

- **Loose equality (`==`)**: Performs type coercion before comparison
- **Strict equality (`===`)**: Checks both type and value without coercion

Falsy Values

Six falsy values in JavaScript: `false`, `0`, `""` (empty string), `null`, `undefined`, and `NaN`.

Scope

- **`var`**: Function-scoped
- **`let` and `const`**: Block-scoped

Typeof Quirks

- `typeof null` returns `"object"` (historical bug)
- `typeof []` returns `"object"` (arrays are objects)
- `typeof NaN` returns `"number"` (special numeric value)

Logical Operators

- **OR (`||`)**: Returns first truthy value or last value
- **AND (`&&`)**: Returns first falsy value or last value
- **NOT (`!`)**: Converts to boolean and negates

Document Created: March 5, 2026
Total Questions: 50 (Questions 51-100)
Topics Covered: Data Types, Operators, Control Flow, Type Coercion, Scope, Loops